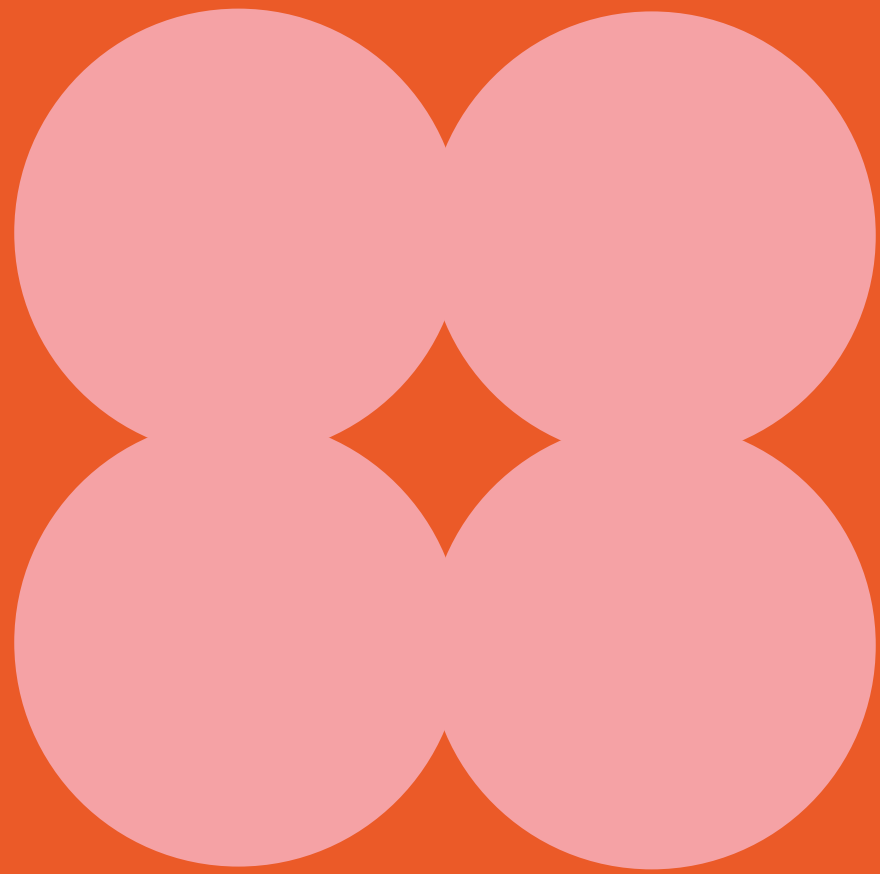




22/01/2026 Tarun sharma

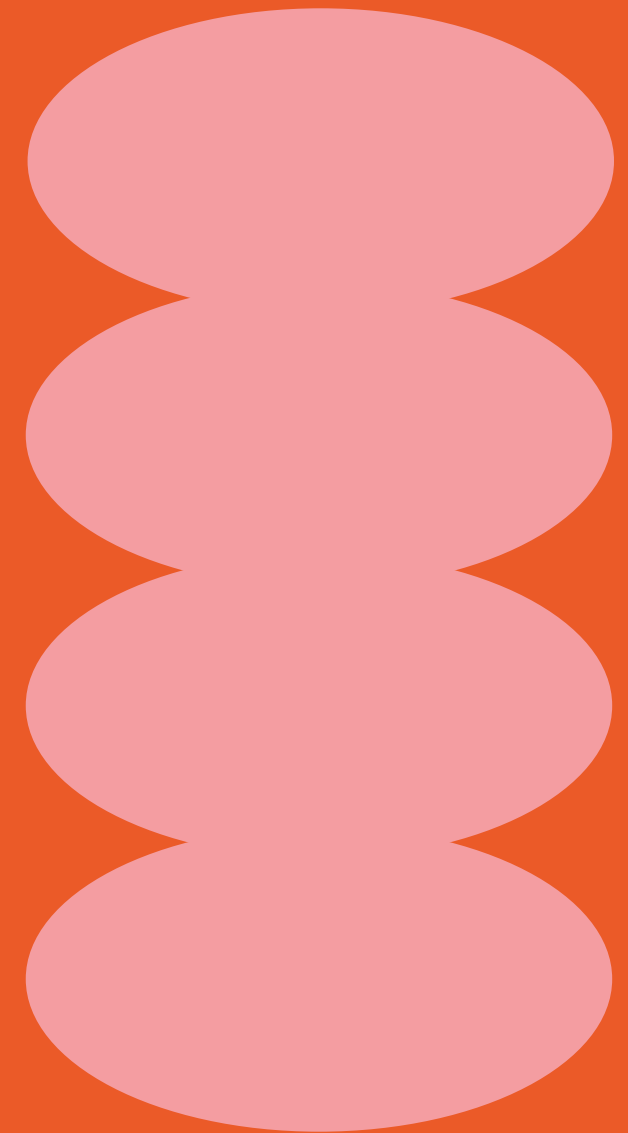
MASTERING DATABASES IN NODE.JS: TYPEORM ESSENTIALS

TypeORM – From Basics to
Production



AGENDA

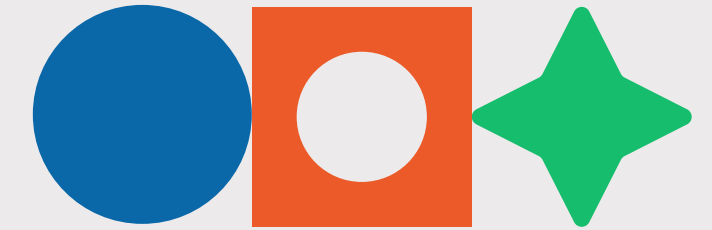
1. Introduction to TypeORM
2. Setup and Configuration
3. Entities and Columns
4. Repository Pattern and Data Operations
5. Entity Relationships and Advanced Queries
6. Transactions, Migrations, and Best Practices
7. Challenges, Use Cases, and Summary





INTRODUCTION TO TYPEORM

WHY TYPEORM?



Widely adopted ORM for Node.js & TypeScript

Streamlines database access with modern JavaScript/TypeScript

Seamless integration with NestJS framework

Fits naturally into enterprise NestJS projects

Supports both Active Record & Data Mapper patterns

Choose the approach that fits your architecture

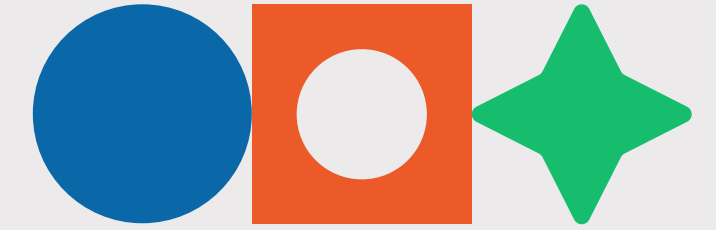
Ideal for enterprise and monolithic applications

Designed for structured, SQL-first development

SQL-first approach with ORM abstraction

Write less SQL, but keep full control when needed

DATABASES TYPEORM SUPPORTS



Broad SQL Database Support

Works with PostgreSQL, MySQL, and MariaDB for robust production use.

Flexible for Development & Testing

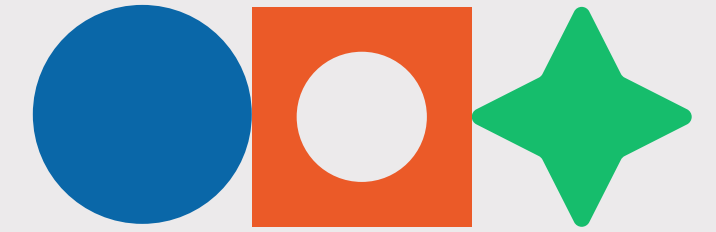
Supports SQLite for lightweight, file-based databases.

Enterprise-Ready Options

Integrates with MSSQL and offers limited Oracle support.



TYPEORM ARCHITECTURE OVERVIEW



Entities map classes to tables

Define data structure using decorators in TypeScript

Repositories handle data access

Encapsulate CRUD operations for each entity

DataSource manages DB connections

Centralizes configuration and connection pooling

Migrations for schema changes

Track and apply database schema updates safely

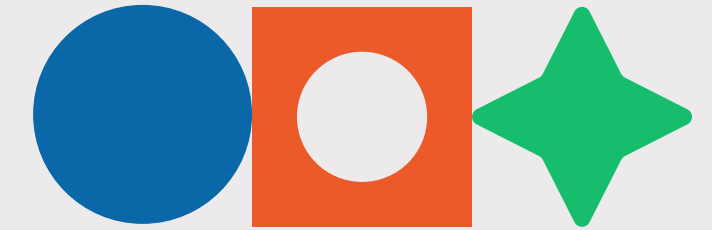
Relations define associations

Use decorators to link entities (e.g., OneToMany)



SETUP AND CONFIGURATION

GETTING TYPEORM INSTALLED



Install TypeORM and Reflect Metadata

Run: `npm install typeorm reflect-metadata`



Add Database Driver

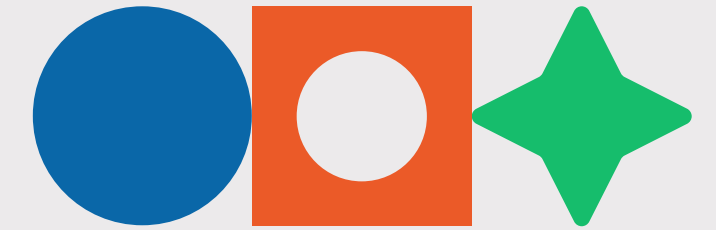
Install driver for your DB (e.g., `pg` for PostgreSQL, `mysql2` for MySQL)



Import Reflect Metadata in Code

Add: `import 'reflect-metadata';` at the top of your entry file

CONFIGURING YOUR DATASOURCE



Configure DataSource with Credentials

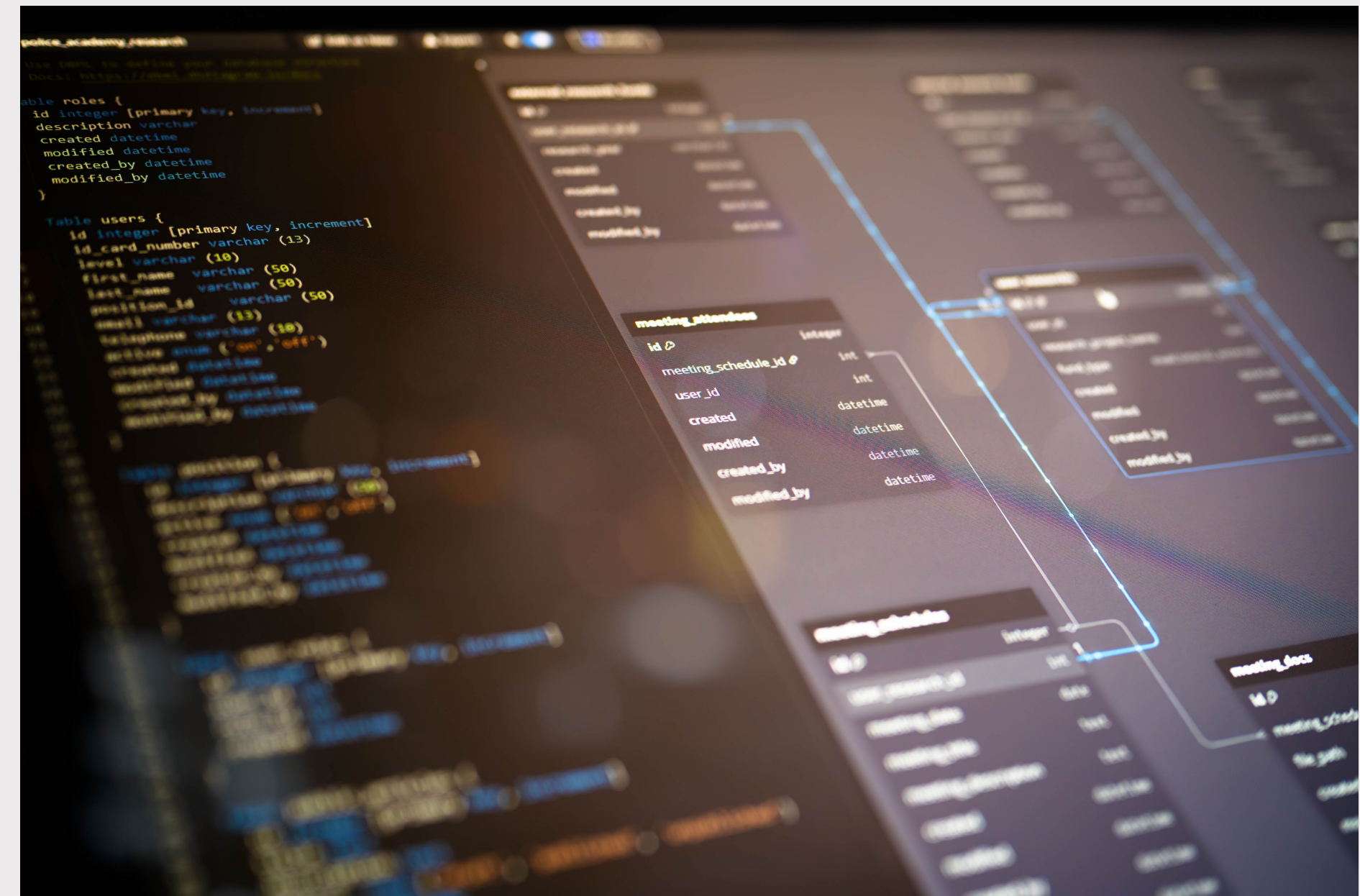
- Specify database type, host, port, username, and password
- Example: PostgreSQL, MySQL, SQLite, etc.

Define Entities and Migrations Paths

- Set file paths for entities and migration scripts
- Ensures TypeORM knows where to find models and migrations

Control Schema Synchronization

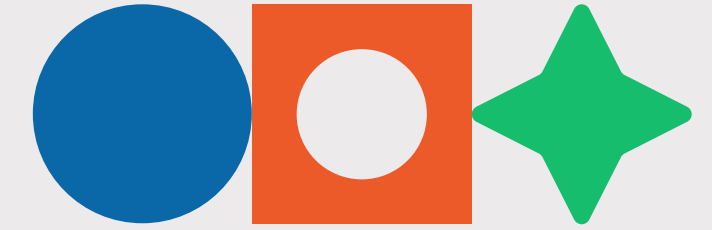
- Use 'synchronize' option for auto schema sync (dev only)
- Always set synchronize: false in production





ENTITIES AND COLUMNS

ENTITIES: MAPPING CLASSES TO TABLES



Class Decorators Define Entities

Use `@Entity` to map a class to a database table.

Properties Become Table Columns

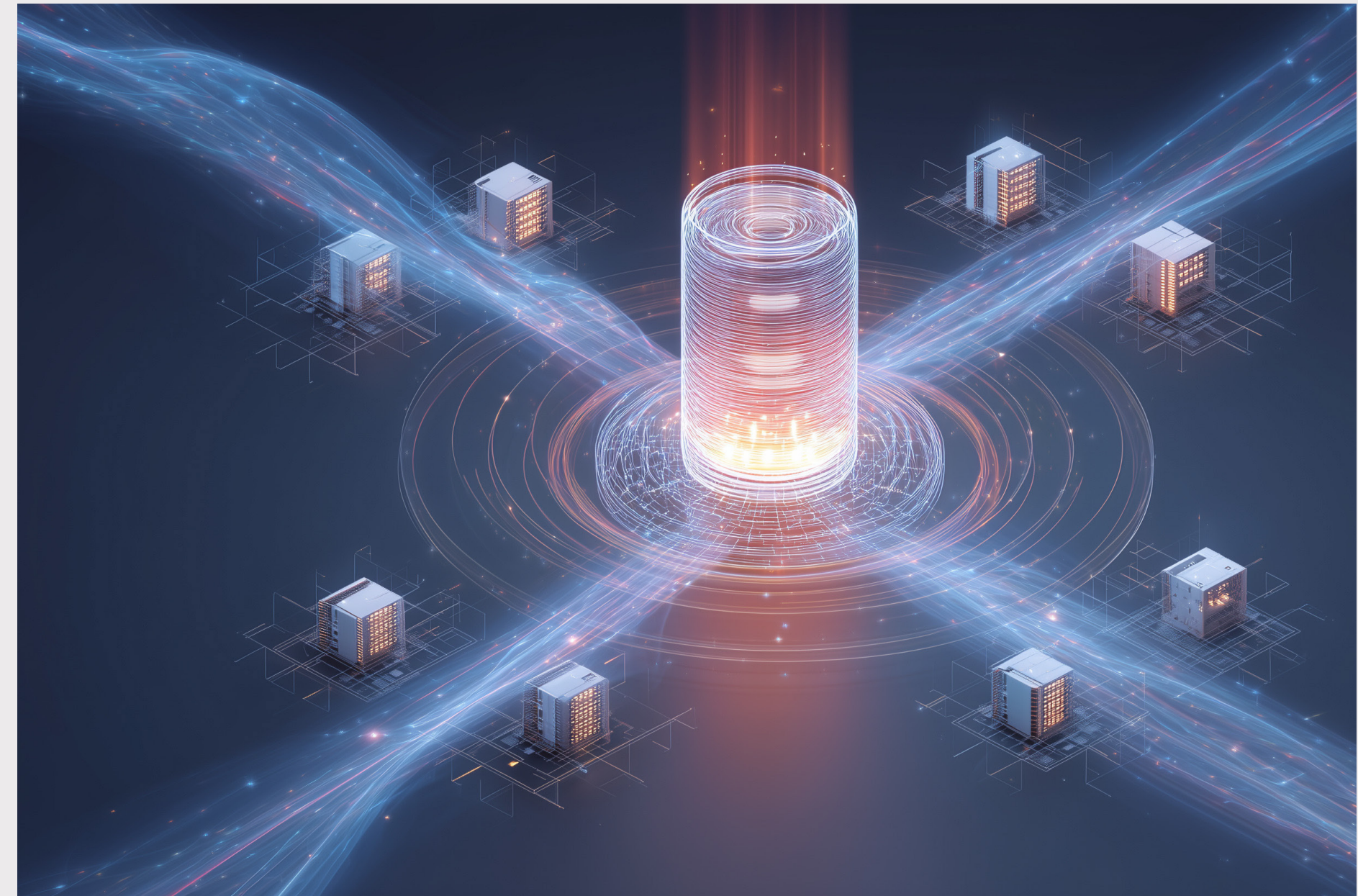
Each property uses `@Column` to specify its mapping.

Customizable Column Options

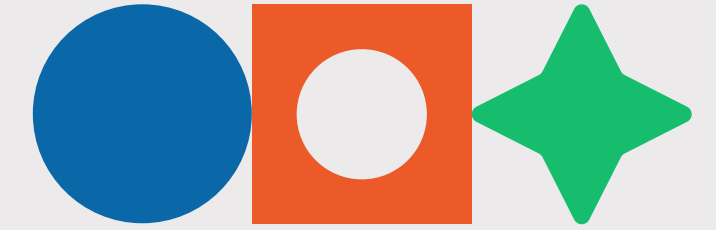
Set options like unique, nullable, default, and length.

Example: User Entity

Showcases decorators and column options in code.



ENTITY EXAMPLE: USER CLASS



Class Decorators Map to Tables

- Use `@Entity` to define a database table

Primary Key with `@PrimaryGeneratedColumn`

- Automatically generates unique IDs for each record

Columns Defined with `@Column`

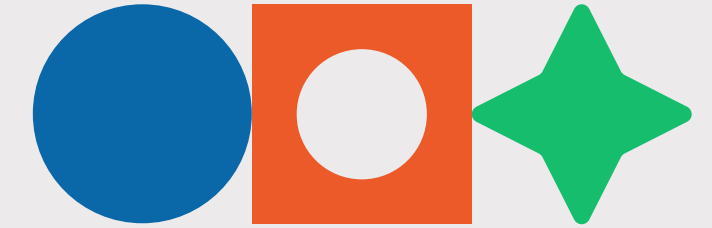
- Add fields like name and email as columns
- Set options: unique, nullable, default values

Example: User Entity

- Simple class with id, name, and unique email



COLUMN TYPES AND OPTIONS



Flexible Column Configuration

- Set columns as nullable or required using the nullable option.
- Assign default values directly in column definitions.
- Control string length and numeric precision for each column.

Support for Multiple Data Types

- Choose from string, number, boolean, date, JSON, and more.

Customizable Columns for Your Needs

- Fine-tune columns with options like unique, default, and length.

Example: Column Options in TypeORM

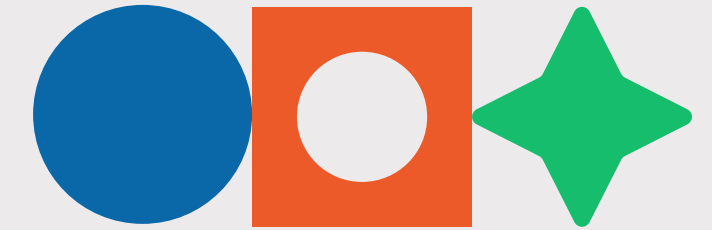
- `@Column({ nullable: true, default: 0, length: 50 })`





REPOSITORY PATTERN AND DATA OPERATIONS

REPOSITORY PATTERN IN TYPEORM



Decouples Data Access from Business Logic

Repositories handle all database operations, keeping business logic clean.



Promotes Testable and Maintainable Code

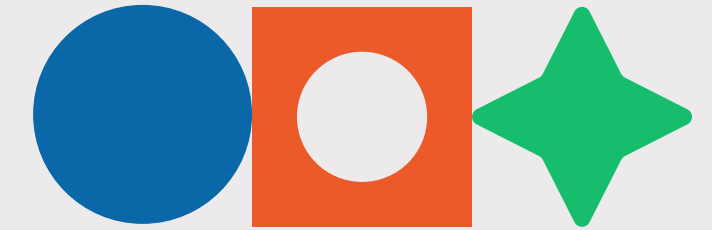
Easier to mock repositories for unit testing and refactor code.



Recommended for Most TypeORM Projects

Encourages best practices and scalable architecture.

CREATING AND SAVING DATA



Build Entities with `repository.create`

- Initialize a new entity instance with desired properties



Persist Data Using `repository.save`

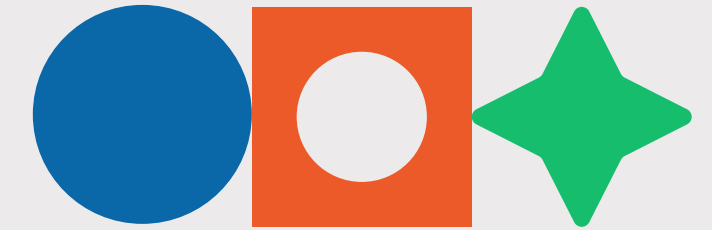
- Save the entity to the database in a single step



Example: Creating a New User

- `const user = userRepo.create({
 name: 'John', email: 'john@test.com'
});`
- `await userRepo.save(user);`

FETCHING DATA FROM THE DATABASE



Retrieve All Records Easily

- Use `repository.find()` to fetch all rows from a table.
- Returns an array of entity objects.

Find a Single Record

- `repository.findOne()` locates a single entity by criteria.
- Ideal for fetching by unique fields like email or ID.

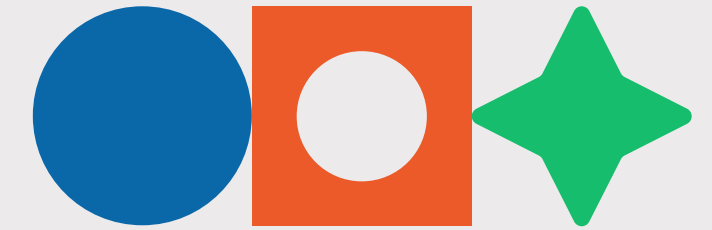
Filter with Where Clauses

- Pass a `where` option to filter results by field values.
- Supports complex queries and multiple conditions.

Example Code

- `const users = await userRepo.find();`
- `const user = await userRepo.findOne({ where: { email: 'john@test.com' } });`

UPDATING AND DELETING RECORDS



Update Records with repository.update

- Modify specific fields by matching criteria, e.g., user ID.
- Does not load the entity before updating.
- Example:
 - `await userRepo.update({ id: 1 }, { name: 'New Name' });`

Delete Records with repository.delete

- Remove records by passing filter criteria.
- Efficient for bulk or single deletions.
- Example:
 - `await userRepo.delete({ id: 1 });`

Target Records by Criteria

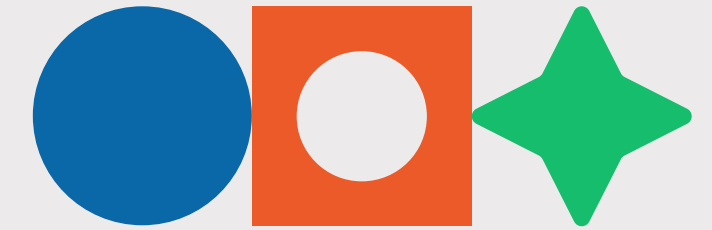
- Use object filters to specify which records to update or delete.
- Supports complex conditions for precise operations.





ENTITY RELATIONSHIPS AND ADVANCED QUERIES

DEFINING ENTITY RELATIONSHIPS



TypeORM Relationship Types

- Supports OneToOne, OneToMany, ManyToOne, ManyToMany

Decorator-Based Setup

- Use @OneToMany, @ManyToOne, etc. to define associations

Example: Company and User

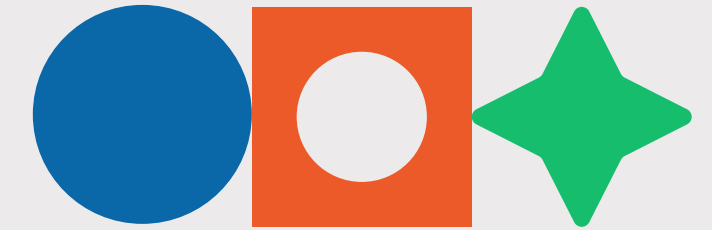
- Company has many Users; User belongs to one Company

Sample Code

- Company entity: @OneToMany(() => User, user => user.company)
- User entity: @ManyToOne(() => Company, company => company.users)



LOADING RELATED DATA



Load Related Data with Relations Option

- Use the 'relations' property in find methods to fetch associated entities.
- Example: Fetch users with their company using relations: ['company'].

Be Cautious with Eager Loading

- Eager loading can cause performance issues, especially in APIs.
- Avoid loading unnecessary relations by default.

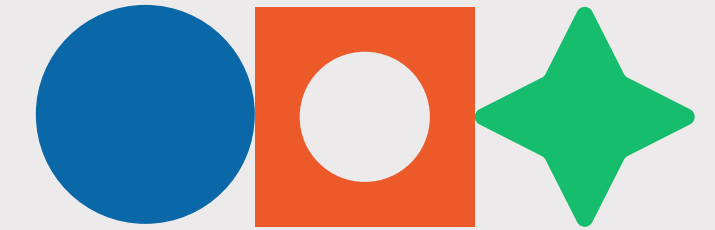
Prevent Performance Pitfalls

- Over-fetching relations may lead to N+1 query problems.
- Always review which relations are loaded to optimize queries.

Code Example: Loading Relations

- ```
const users = await userRepo.find({ relations: ['company'] });
```

# ADVANCED QUERIES WITH QUERYBUILDER



## Fluent SQL Query Construction

- Use QueryBuilder for readable, chainable query syntax
- Easily compose WHERE, ORDER BY, and JOIN clauses

## Flexible Filtering and Sorting

- Apply dynamic filters and sorting at runtime
- Supports complex conditions and parameter binding

## Retrieve Results Efficiently

- Use `getMany()` for lists, `getOne()` for single records
- Optimized for performance and clarity

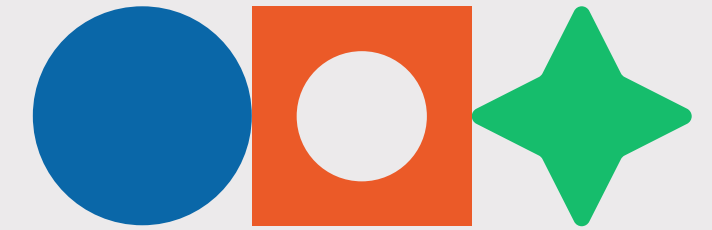
## Example: QueryBuilder in Action

- `const users = await userRepo`
- `.createQueryBuilder('user')`
- `.where('user.isActive = :active', { active: true })`
- `.orderBy('user.createdAt', 'DESC')`
- `.getMany();`



# **TRANSACTIONS, MIGRATIONS, AND BEST PRACTICES**

# HANDLING TRANSACTIONS



## Atomic Operations with Transactions

Wrap multiple DB actions in a single transaction block

## Access Repositories Within Transaction

Use the transaction manager to get repositories

## Ensure Data Consistency

All changes succeed or all are rolled back on error

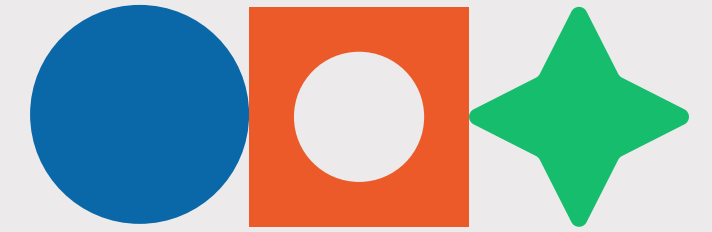
## Code Example

```
await AppDataSource.transaction(async manager => {
 const userRepo = manager.getRepository(User);
 const orderRepo = manager.getRepository(Order);
 const user = await userRepo.save({ name: 'John' });
 await orderRepo.save({ userId: user.id });
});
```





# MANAGING SCHEMA WITH MIGRATIONS



## Never Use Synchronize in Production

Avoid automatic schema sync to prevent data loss



## Migrations Track Schema Changes

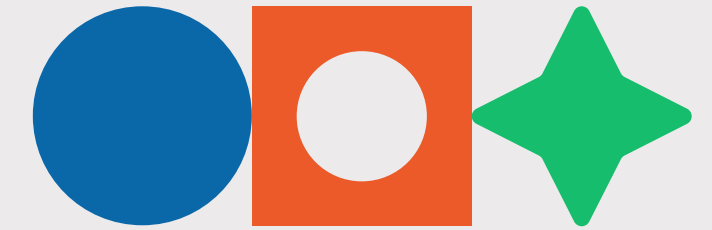
Each migration records a specific DB structure update



## Version Control for Your Database

Migrations provide a history and rollback path

# BEST PRACTICES FOR TYPEORM



## **Adopt the Data Mapper Pattern**

Keep database logic in repositories, not in entities

## **Design Simple, Focused Entities**

Entities should only define table structure and relations

## **Move Business Logic to Services**

Keep entities free of business rules for maintainability

## **Always Use Migrations**

Track and apply schema changes safely in all environments

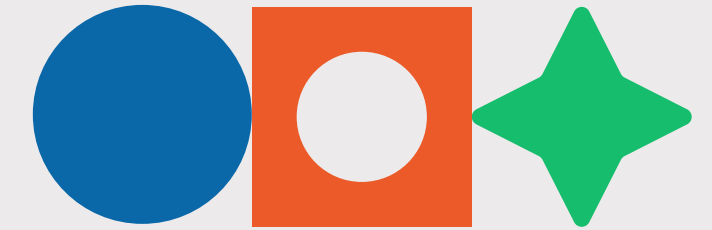
A large decorative graphic on the right side of the slide. It features several overlapping, rounded, organic shapes in a variety of colors including teal, green, yellow, orange, red, and pink. Overlaid on these shapes is the text "Best Practices" in a large, white, bold, sans-serif font.

**Best Practices**



# **CHALLENGES, USE CASES, AND SUMMARY**

# COMMON PITFALLS TO AVOID



## N+1 Query Problems

Fetching related data in loops causes excessive DB queries

## Over-fetching Relations

Loading unnecessary related entities impacts performance

## Tight Entity-API Coupling

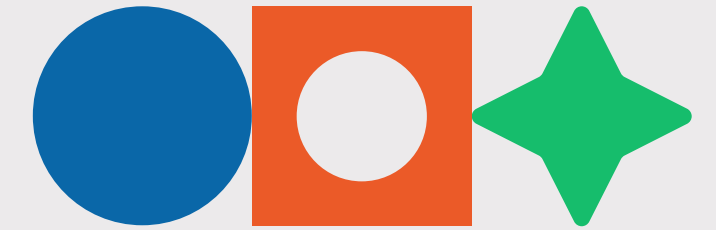
Directly exposing entities as API models reduces flexibility

## Business Logic in Entities

Placing heavy logic inside entities makes code hard to maintain



# WHEN TYPEORM SHINES



## Excels in NestJS and SQL Environments

Seamless integration with NestJS and SQL databases

## Ideal for Enterprise & Monolithic Apps

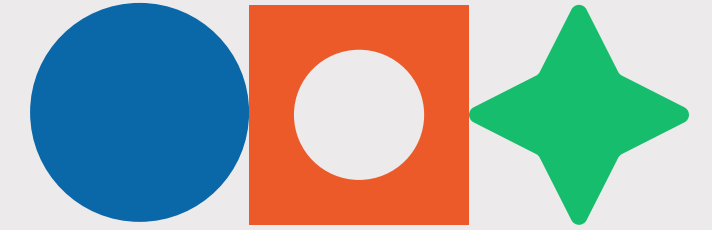
Supports complex, structured, and modular monolith architectures

## Not Suited for Serverless or Edge-first

Less effective in stateless, distributed, or serverless deployments



# KEY TAKEAWAYS



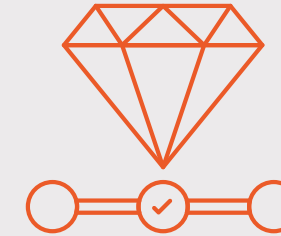
## Robust SQL ORM for Node.js

TypeORM provides a powerful abstraction over SQL databases.



## Ideal for Structured, Enterprise Apps

Best suited for large, organized, and scalable applications.



## Emphasize Best Practices

Use migrations, keep entities simple, and separate business logic.